

Tnt et slimes



Problème dans notre version actuelle:

pas de réaction en chaîne sur les tnt:

1 Une TNT explose

2 Elle détruit les blocs autour d'elle

3 Les autres TNT proches ? Ignorées ❌

```
public void Explode(Vector3 origin)
{
    int radius = 3; // rayon de l'explosion
    // boucle x, y, z ...
    if (offset.sqrMagnitude > radius * radius) continue;
    if (!blocks.ContainsKey(pos)) continue;
    blocksToDestroy.Add(pos); // ← pas de vérif TNT !
    DestroyMultipleBlocks(blocksToDestroy);
}
```

Solution: fonction recursive

une nouvelle méthode récursive qui propage les explosions.

```
private void CascadeExplosion(int radius, Vector3Int center,
                             HashSet<Vector3Int> blocksToDestroy, HashSet<Vector3Int> tnt)
{
    // boucle x, y, z → même logique qu'avant ...
    if (blocks[pos] == BlockType.Tnt && !tnt.Contains(pos))
    {
        tnt.Add(pos);           // ✅ on la marque
        CascadeExplosion(radius, pos, blocksToDestroy, tnt); // 🔁
    }
    blocksToDestroy.Add(pos);
}
```

tnt.Contains(pos) → si déjà vue : on saute. Anti-boucle infinie !

CascadeExplosion(...) → la fonction se rappelle elle-même (récursivité).

C'est quoi la récursivité

=> Une fonction qui s'appelle elle-même.

Les poupées russes

Tu ouvres une poupée → il y en a une dedans →
tu l'ouvres → encore une → ... jusqu'à la plus
petite.

Dans notre jeu

TNT explose → cherche des TNT proches →
déclenche la même fonction pour chacune.

Condition d'arrêt :

Sans elle, la fonction s'appelle à l'infini → le jeu plante !
dans notre jeu la condition d'arrêt est de ne plus trouver de tnt déjà connue dans le radius

Explode() - après

Elle orchestre la cascade, puis détruit tout d'un coup.

```
public void Explode(Vector3 origin)
{
    var tnt = new HashSet<Vector3Int>();
    tnt.Add(center); // TNT de départ

    CascadeExplosion(radius, center, blocksToDestroy, tnt); // 🔄 lance la cascade

    foreach (Vector3Int pos in tnt) // 💣 une explosion visuelle par TNT
    {
        Instantiate(explosionPrefab, pos, ...);
    }
    DestroyMultipleBlocks(blocksToDestroy); // tout détruire à la fin
}
```

Explode et CascadeExplosion

voici les deux fonctions au complet

1 reference

```
public void Explode(Vector3 origin)
{
    int radius = 3;

    Vector3Int center = Vector3Int.FloorToInt(origin);

    HashSet<Vector3Int> blocksToDestroy = new HashSet<Vector3Int>();
    HashSet<Vector3Int> tnt = new HashSet<Vector3Int>();
    tnt.Add(Vector3Int.FloorToInt(origin));
    CascadeExplosion(radius, center, blocksToDestroy, tnt);

    foreach (Vector3Int pos in tnt)
    {
        ParticleSystem vfx = Instantiate(explosionPrefab, pos, Quaternion.identity);
        vfx.Play();
        Destroy(vfx.gameObject, vfx.main.duration + vfx.main.startLifetime.constantMax);
    }

    DestroyMultipleBlocks(blocksToDestroy);
}
```

```
private void CascadeExplosion(int radius, Vector3Int center, HashSet<Vector3Int> blocksToDestroy, HashSet<Vector3Int> tnt)
{
    for (int x = -radius; x <= radius; x++)
    {
        for (int y = -radius; y <= radius; y++)
        {
            for (int z = -radius; z <= radius; z++)
            {
                Vector3Int offset = new Vector3Int(x, y, z);
                Vector3Int pos = center + offset;

                // sphere check (distance² <= radius²)
                if (offset.sqrMagnitude > radius * radius)
                    continue;

                // only destroy existing blocks
                if (!blocks.ContainsKey(pos))
                    continue;

                if (blocks[pos] == BlockType.Tnt && !tnt.Contains(pos))
                {
                    tnt.Add(pos);
                    CascadeExplosion(radius, pos, blocksToDestroy, tnt);
                    Debug.Log("will kaboom at pos" + pos);
                }

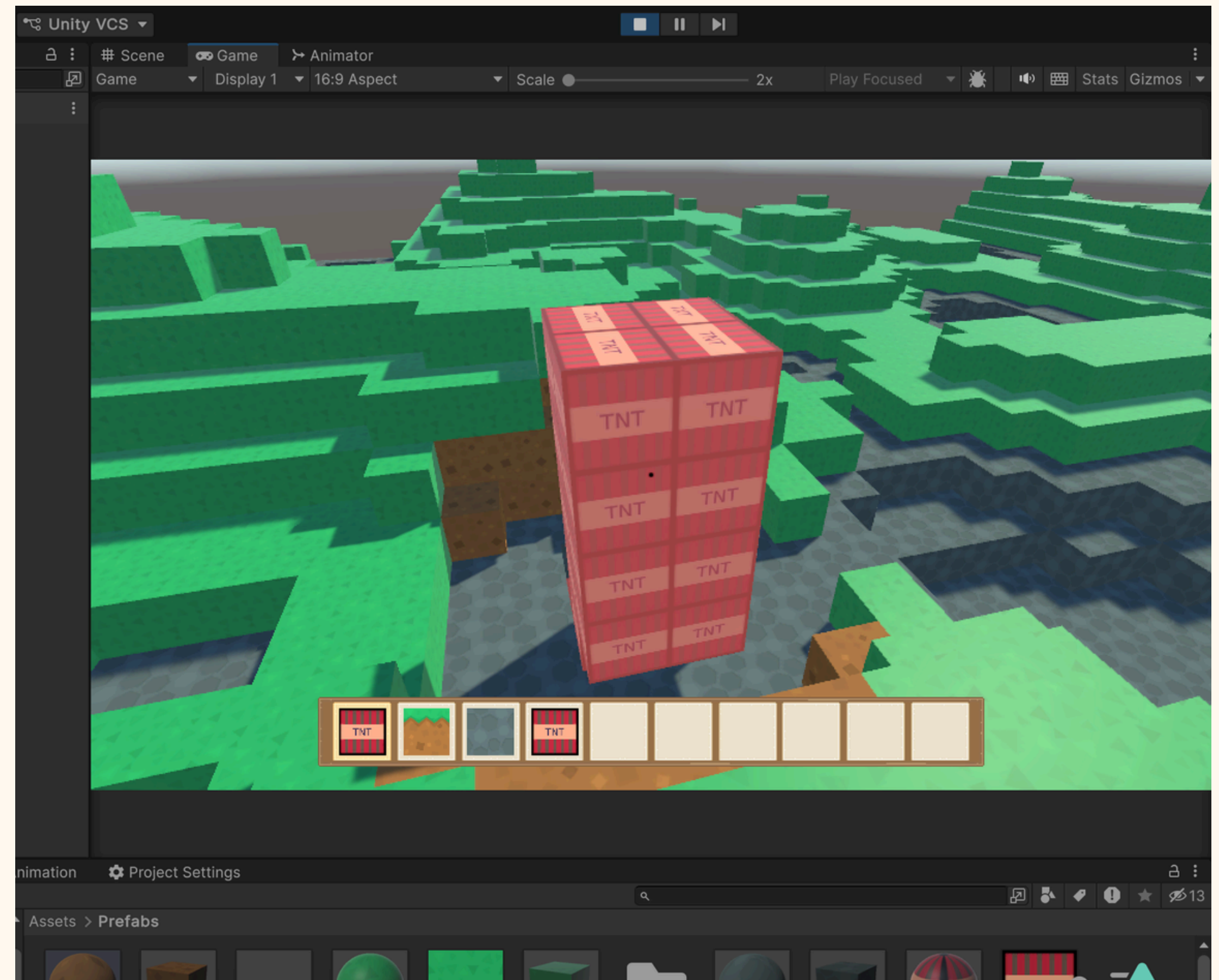
                blocksToDestroy.Add(pos);
            }
        }
    }
}
```

Faites exploser des tnt dans le même radius



Animation de TNT.

- **Objectif**
 - Ajouter une TNT “allumée” comme Minecraft.
 - Toutes les TNT de la cascade s’allument en même temps.
 - Les animations démarrent ensemble, puis explosion après délai.

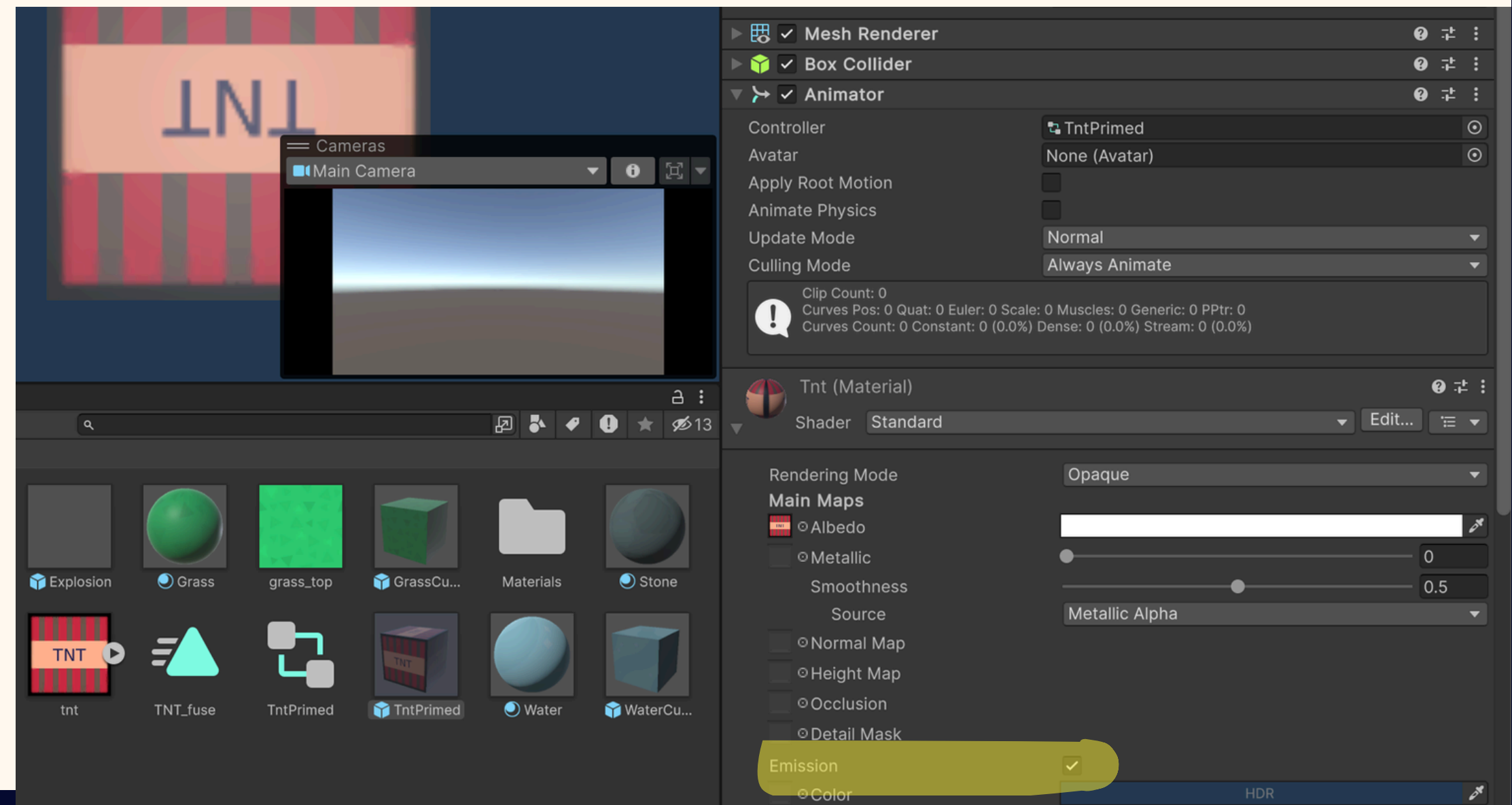


- **Principe**

- **Le bloc voxel TNT n'a pas d'Animator.**
- **Quand on appuie sur E, on va remplacer chaque bloc TNT concerné par un prefab visuel TntPrimed.**
- **Ce prefab joue l'animation de mèche en boucle.**
- **Après fuseTime, tout explose en même temps.**

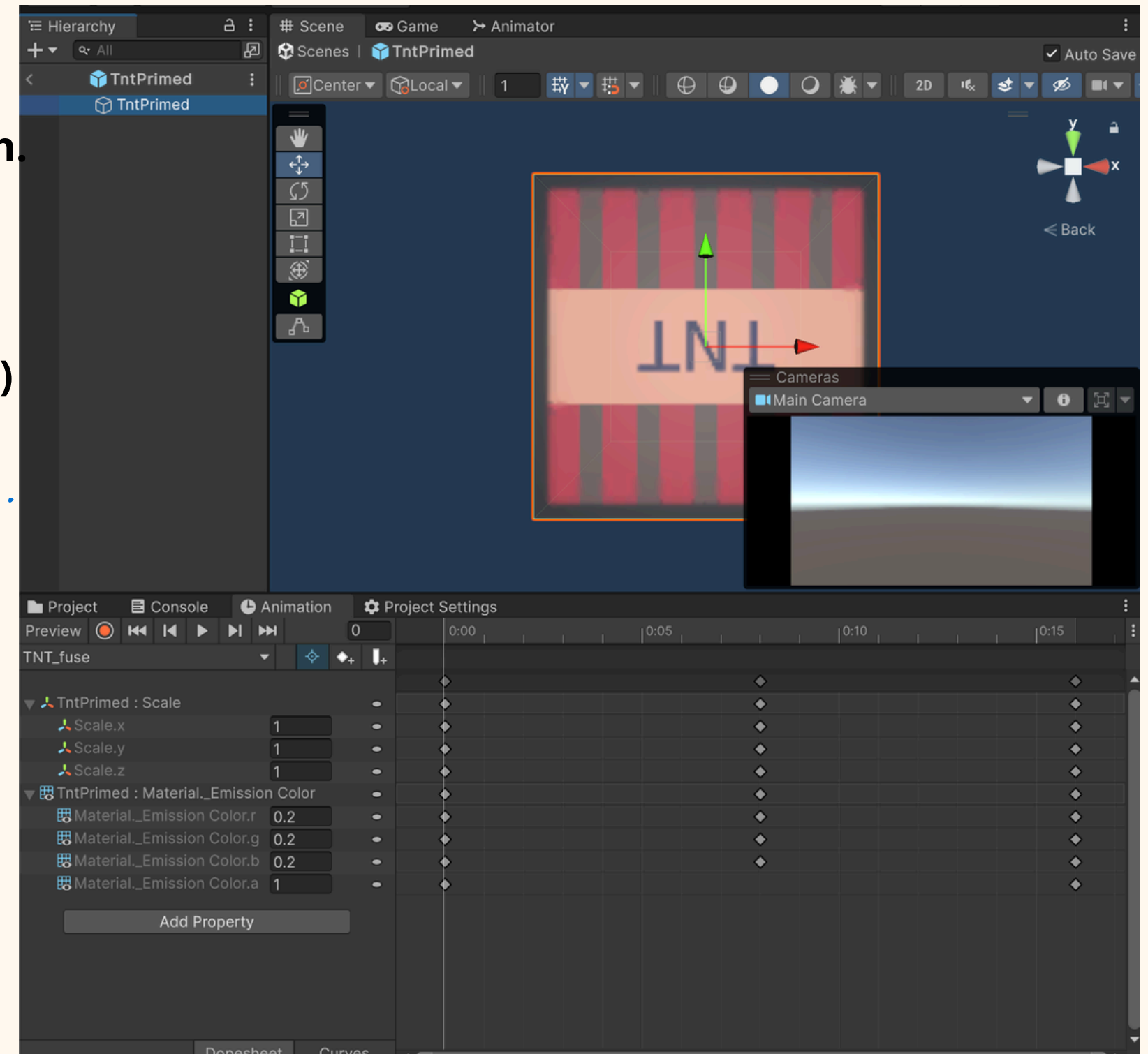
Création du Prefab TntPrimed

- Dupliquez un de vos prefab de cube.
- Nommez le TntPrimed.
- Ajoutez le material de tnt
- cochez Emission sur le material
- Ajoutez Animator sur TntPrimed.
- Glissez l'objet dans Assets/Prefabs pour créer le prefab.



Création de l'animation

- Ouvrir Window > Animation > Animation.
- Sélectionnez TntPrimed, cliquer Create, nommer TNT_fuse.anim.
- Ajouter propriétés Scale et Material._EmissionColor.
- Valeurs d'animation
 - Temps 0:00 : Scale = 1,1,1 ; Emission = (0.02, 0.02, 0.02, 1)
 - Temps 0:08 : Scale = 1.08,1.08,1.08 ; Emission = (1.50, 0.35, 0.35, 1)
 - Temps 0:16 : Scale = 1,1,1 ; Emission = (0.02, 0.02, 0.02, 1)

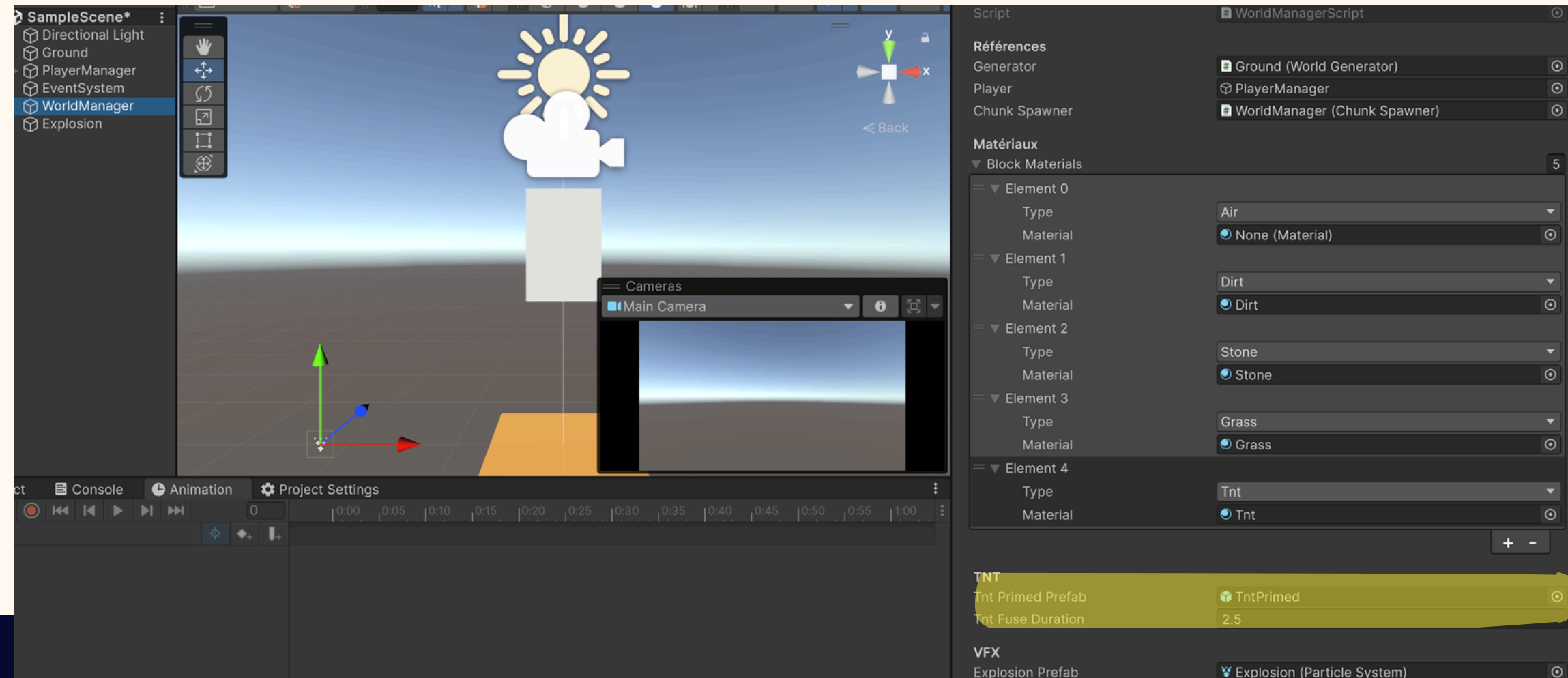


Référence dans le WorldManager

- Dans WorldManagerScript.cs, ajoutez :

```
[Header("TNT")]  
2 references | Unity Serialized Field  
public GameObject tntPrimedPrefab;  
1 reference | Unity Serialized Field  
public float tntFuseDuration = 2.5f;
```

- Dans l'Inspector, assigner tntPrimedPrefab.



Helper de suppression sans rebuild

- Dans WorldManagerScript.cs, ajoutez :

```
1 reference
private void RemoveBlockNoRebuild(Vector3Int p)
{
    if (!blocks.ContainsKey(p))
        return;

    blocks.Remove(p);
    chunkSpawner.OnBlockDestroyed(p);
}
```


Détection de la cascade + lancement

- But: trouver toutes les TNT concernées et préparer les blocs à détruire.
- Ensuite on lance une coroutine pour gérer le délai de mèche.
- modifiez votre fonction Explode

```
1 reference
public void Explode(Vector3 origin)
{
    int radius = 3;
    Vector3Int center = Vector3Int.FloorToInt(origin);

    HashSet<Vector3Int> blocksToDestroy = new HashSet<Vector3Int>();
    HashSet<Vector3Int> tnt = new HashSet<Vector3Int>();
    tnt.Add(center);

    CascadeExplosion(radius, center, blocksToDestroy, tnt);

    // Toutes les TNT de la cascade partent ensemble
    StartCoroutine(IgniteThenExplode(blocksToDestroy, tnt));
}
```

IgniteThenExplode

- IgniteThenExplode est une coroutine:
- Elle allume toutes les TNT de la cascade dans la même frame (animation synchronisée).
- Elle attend `tntFuseDuration`.
- Elle fait exploser tout en même temps.

```
1 reference
private IEnumerator IgniteThenExplode(HashSet<Vector3Int> blocksToDestroy, HashSet<Vector3Int> tnt)
{
    List<GameObject> primedInstances = new List<GameObject>();

    // 1) allumage simultané (même frame)
    foreach (Vector3Int pos in tnt)
    {
        if (!blocks.TryGetValue(pos, out var type) || type != BlockType.Tnt)
            continue;

        RemoveBlockNoRebuild(pos);

        if (tntPrimedPrefab != null)
            primedInstances.Add(Instantiate(tntPrimedPrefab, pos + Vector3.one * 0.5f, Quaternion.identity));
    }

    RebuildChunk();

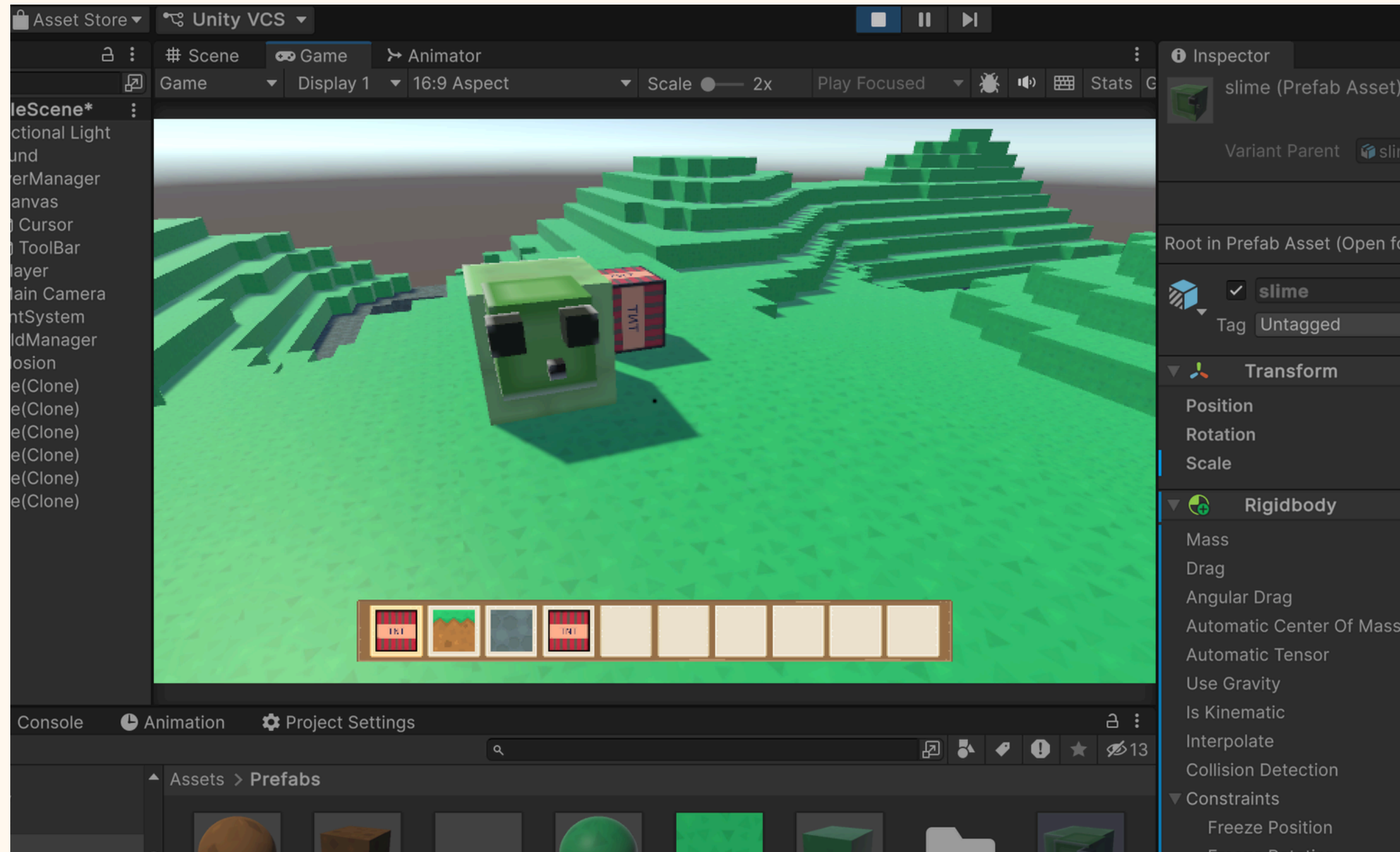
    // 2) délai commun
    yield return new WaitForSeconds(tntFuseDuration);

    // 3) explosion simultanée
    foreach (GameObject primed in primedInstances)
    {
        if (primed != null)
            Destroy(primed);
    }

    foreach (Vector3Int pos in tnt)
    {
        ParticleSystem vfx = Instantiate(explosionPrefab, pos, Quaternion.identity);
        vfx.Play();
        Destroy(vfx.gameObject, vfx.main.duration + vfx.main.startLifetime.constantMax);
    }

    DestroyMultipleBlocks(blocksToDestroy);
}
```

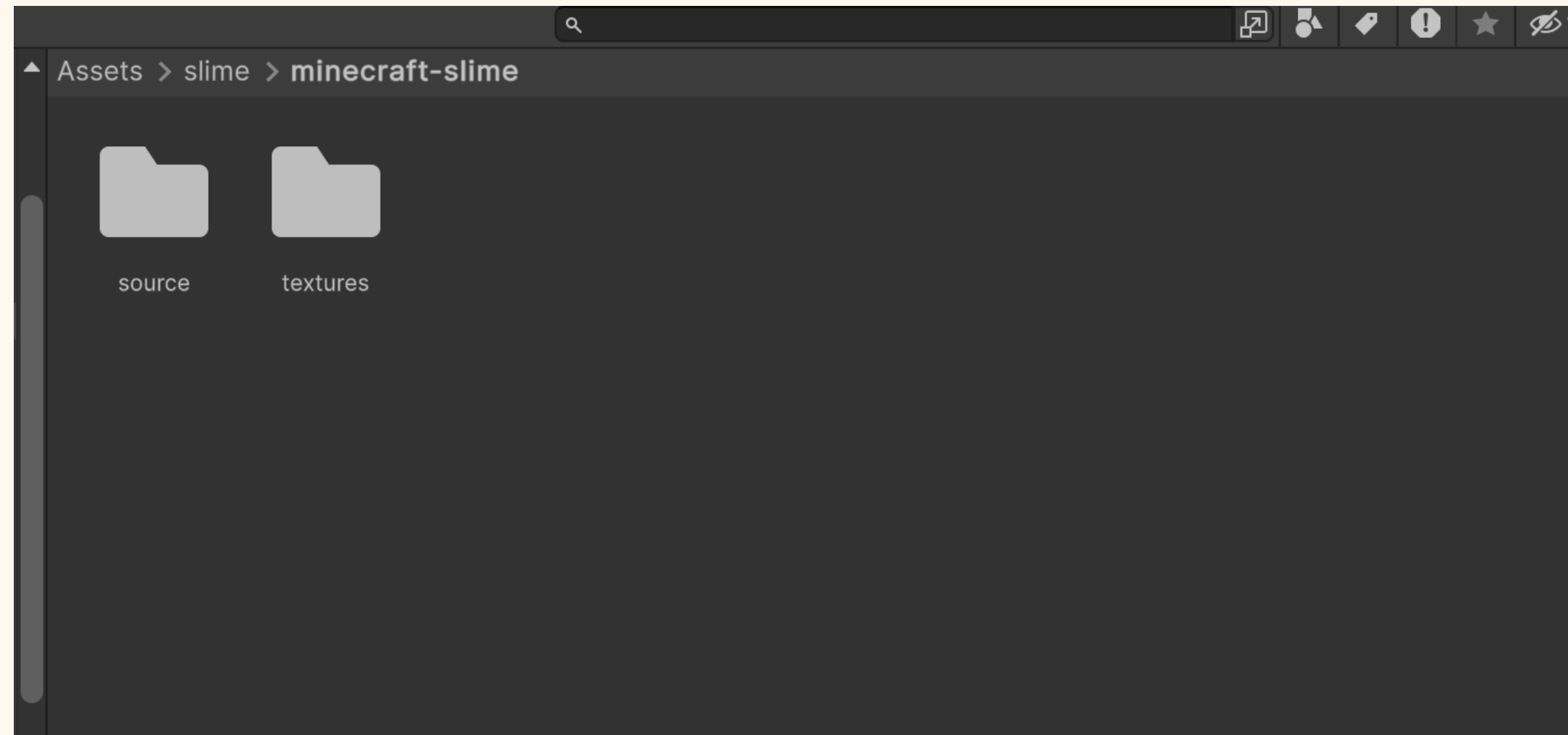
Maintenant: un slime !



Télécharger l'asset

- Téléchargez le fichier ZIP.
- Décompressez-le.
- Créez un nouveau dossier Slime dans le dossier Assets sur unity.
- Glissez le contenu du ZIP dans ce dossier.
- lien de téléchargement:

<https://github.com/Ayly-EXE/LogiCube/blob/Cours6/MultipleTntAndSlime/minecraft-slime.zip>

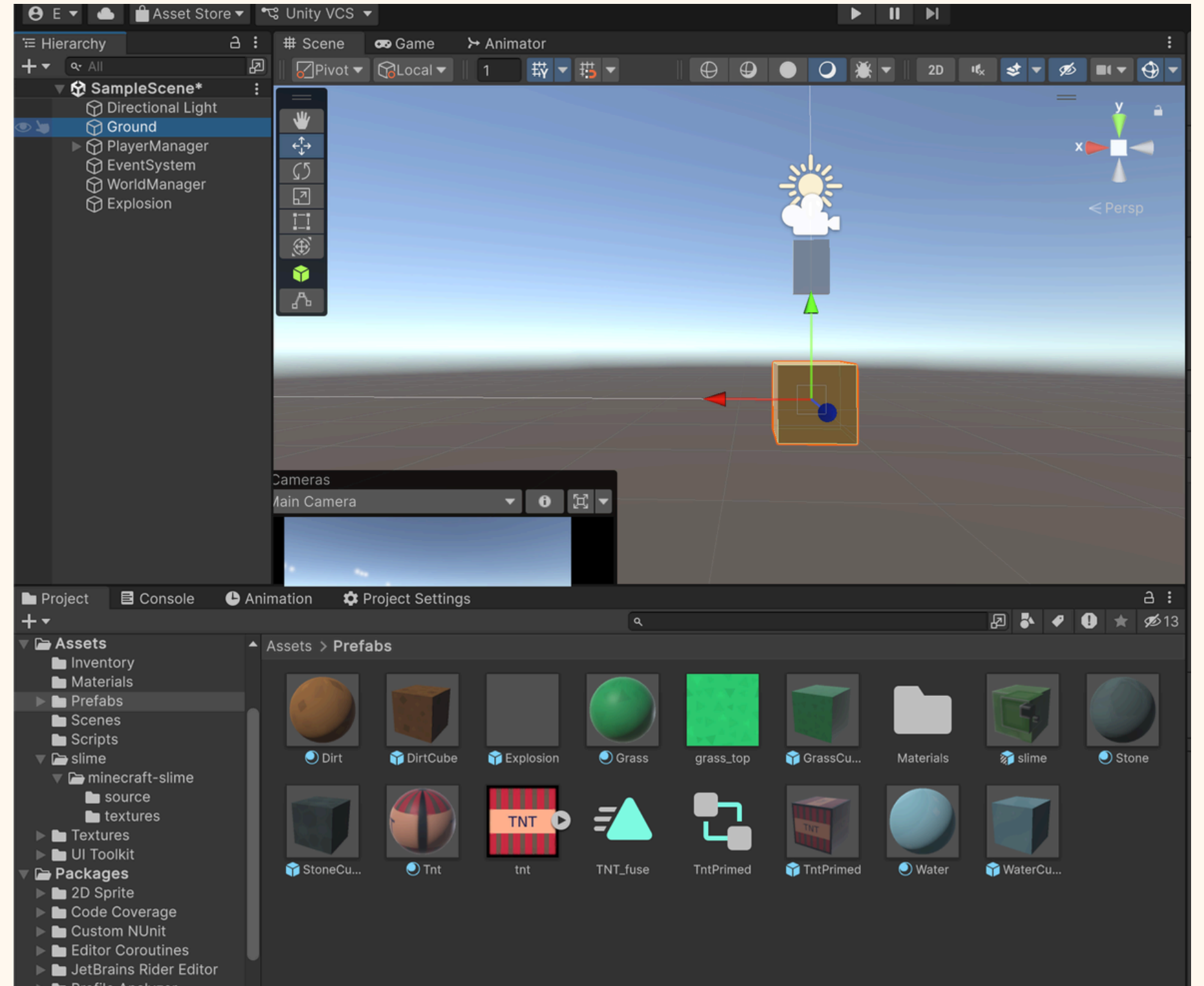


- Glissez le slime depuis le dossier Source dans la scène.
- Réglez son Scale sur (7, 7, 7).
- Ajoutez un composant Rigidbody.
- Ajoutez un composant Box Collider.
- Réglez la taille du Box Collider sur (0.2032, 0.2032, 0.2032).



Créer un prefab

- Glissez le slime de votre Hierarchy vers le dossier Prefabs.
- supprimez-le de la scène.



SlimeController

- Créez un nouveau script SlimeController.
- Le script SlimeController.cs gère tout le comportement du slime.
- Il fait 3 choses: déplacement aléatoire, saut, division en 2 quand on le tape.
- Le slime se divise jusqu'à une limite (`maxGenerations = 5`), puis il disparaît.

SlimeController

- Le script est long.
- Ne faites pas tout à la main.
- Le code complet est dans Git, vous pouvez le copier/coller depuis le repo.
- En présentation, on se concentre sur les fonctions principales et leur rôle.

Fonctions de cycle

- Awake()
 - récupère le Rigidbody,
 - applique les contraintes physiques,
 - initialise une première direction,
 - lance les timers de changement de direction et de saut.
- Update()
 - vérifie si c'est le moment de changer de direction,
 - vérifie si c'est le moment de sauter,
 - déclenche ces actions selon les timers.
- FixedUpdate()
 - applique la vitesse horizontale au Rigidbody,
 - garde un mouvement fluide et stable avec la physique Unity.

```
0 references | Unity Message
void Awake()
{
    rb = GetComponent<Rigidbody>();
    rb.interpolation = RigidbodyInterpolation.Interpolate;
    rb.constraints |= RigidbodyConstraints.FreezeRotationX | RigidbodyConstraints.FreezeRotationZ;

    if (maxDirectionChangeDelay < minDirectionChangeDelay)
        maxDirectionChangeDelay = minDirectionChangeDelay;
    if (maxJumpDelay < minJumpDelay)
        maxJumpDelay = minJumpDelay;

    PickNewDirection(force: true);
    ScheduleNextDirectionChange();
    ScheduleNextJump();
}
```

TakeHit()

- empêche les doubles hits trop rapides (cooldown),
- vérifie si le slime peut encore se diviser,
- si oui: appelle la division,
- puis détruit le slime parent.

1 reference

```
public void TakeHit()
{
    if (Time.time < nextHitAllowedAt)
        return;

    nextHitAllowedAt = Time.time + hitCooldown;

    if (generation >= maxGenerations)
    {
        Destroy(gameObject);
        return;
    }

    SpawnChildren();
    Destroy(gameObject);
}
```


SpawnChildren()

- calcule la génération suivante,
- réduit l'échelle des enfants,
- crée 2 enfants à gauche et à droite du parent,
- donne aux enfants une direction initiale et une petite impulsion,
- augmente légèrement leur vitesse pour garder du rythme.
- Résultat in-game: chaque hit crée deux slimes plus petits et plus dynamiques.

```
private void SpawnChildren()
{
    int childGeneration = generation + 1;
    Vector3 childScale = transform.localScale * childScaleMultiplier;

    Vector3 forward = moveDirection.sqrMagnitude > 0.001f ? moveDirection : transform.forward;
    Vector3 side = Vector3.Cross(Vector3.up, forward).normalized;
    if (side.sqrMagnitude < 0.001f)
        side = Vector3.right;

    for (int i = -1; i <= 1; i += 2)
    {
        Vector3 spawnPos = transform.position + side * (i * childSpawnOffset) + Vector3.up * childUpOffset;
        GameObject childObject = Instantiate(gameObject, spawnPos, Quaternion.identity);

        SlimeController child = childObject.GetComponent<SlimeController>();
        if (child != null)
        {
            child.generation = childGeneration;
            child.transform.localScale = childScale;
            child.moveSpeed = moveSpeed * 1.08f;
            child.nextHitAllowedAt = Time.time + 0.12f;
            child.PickNewDirection(force: true);
            child.ScheduleNextJump();
        }

        if (childObject.TryGetComponent<Rigidbody>(out Rigidbody childRb))
        {
            childRb.velocity = Vector3.zero;
            Vector3 impulseBasis = child != null ? child.moveDirection : forward;
            Vector3 impulseDir = (impulseBasis + Vector3.up * 0.2f).normalized;
            childRb.AddForce(impulseDir * splitImpulse, ForceMode.VelocityChange);
        }
    }
}
```

Déplacement aléatoire

PickNewDirection(force)

- tire une direction aléatoire sur le plan XZ,
- évite les directions nulles,
- lisse le changement avec un "jitter" pour éviter les virages trop brutaux.
- ScheduleNextDirectionChange()
- choisit un délai aléatoire entre min/max,
- programme le prochain changement.

```
private void PickNewDirection(bool force)
{
    Vector3 randomDir = new Vector3(Random.Range(-1f, 1f), 0f, Random.Range(-1f, 1f));
    if (randomDir.sqrMagnitude < 0.04f)
        randomDir = transform.forward;
    randomDir.Normalize();

    if (force || moveDirection.sqrMagnitude < 0.001f)
    {
        moveDirection = randomDir;
        return;
    }

    moveDirection = Vector3.Lerp(moveDirection, randomDir, directionJitter).normalized;
}
```

SlimeSpawner

- Créez un nouveau script SlimeSpawner.cs
- ajoutez ces variables d'instances
 - slimePrefab : référence du prefab à instancier quand on spawn.
 - spawnOrigin : transform utilisé comme point de départ (joueur)
 - spawnKey : touche clavier qui déclenche le spawn (par défaut P).
 - spawnDistance : distance devant spawnOrigin où le slime apparaît.
 - spawnHeight : hauteur ajoutée pour éviter un spawn dans le sol.
 - maxAliveSlimes : nombre max de slimes autorisés en même temps

```
[Header("Spawn")]  
2 references | Unity Serialized Field  
public SlimeController slimePrefab;  
2 references | Unity Serialized Field  
public Transform spawnOrigin;  
1 reference | Unity Serialized Field  
public KeyCode spawnKey = KeyCode.P;  
1 reference | Unity Serialized Field  
[Min(0f)] public float spawnDistance = 4f;  
1 reference | Unity Serialized Field  
[Min(0f)] public float spawnHeight = 1.5f;  
1 reference | Unity Serialized Field  
[Min(1)] public int maxAliveSlimes = 80;
```

SlimeSpawner

- ajoutez ces deux fonctions

- vérifie la limite maxAliveSlimes,
- calcule la position de spawn depuis spawnOrigin,
- fait un raycast vers le bas pour poser le slime proprement sur le terrain,
- instancie le prefab à cette position.

```
0 references | Unity Message  
void Update()  
{  
    if (Input.GetKeyDown(spawnKey))  
        SpawnSlime();  
}
```

```
public void SpawnSlime()  
{  
    if (FindObjectsByType<SlimeController>(FindObjectsSortMode.None).Length >= maxAliveSlimes)  
        return;  
  
    Transform origin = spawnOrigin != null ? spawnOrigin : transform;  
    Vector3 spawnPos = origin.position + origin.forward * spawnDistance + Vector3.up * spawnHeight;  
  
    if (Physics.Raycast(spawnPos + Vector3.up * 30f, Vector3.down, out RaycastHit hit, 80f))  
        spawnPos = hit.point + Vector3.up * 0.55f;  
  
    Instantiate(slimePrefab, spawnPos, Quaternion.identity);  
}
```

SlimeSpawner

- ajoutez ces deux fonctions

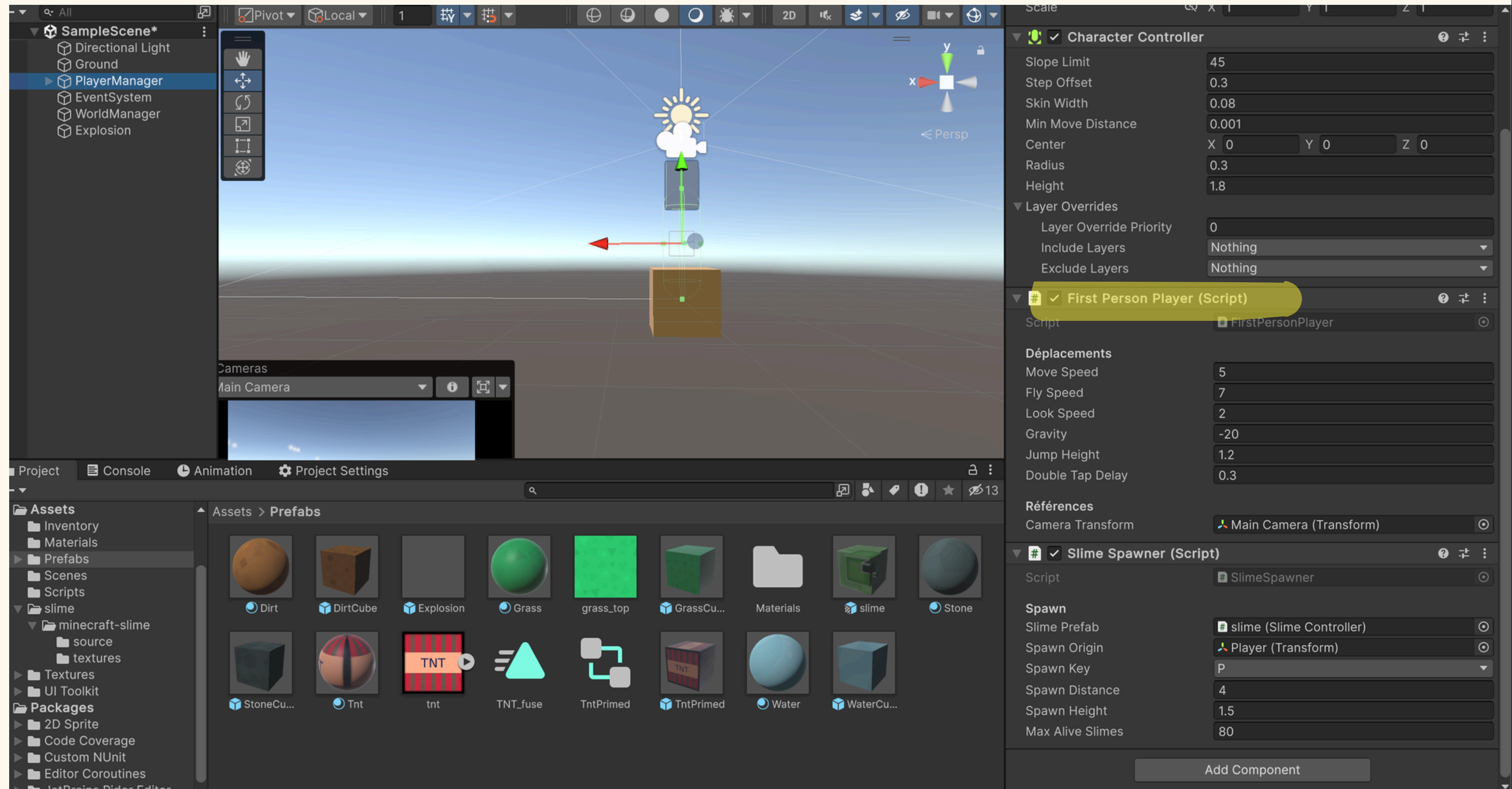
- vérifie la limite maxAliveSlimes,
- calcule la position de spawn depuis spawnOrigin,
- fait un raycast vers le bas pour poser le slime proprement sur le terrain,
- instancie le prefab à cette position.

```
0 references | Unity Message  
void Update()  
{  
    if (Input.GetKeyDown(spawnKey))  
        SpawnSlime();  
}
```

```
public void SpawnSlime()  
{  
    if (FindObjectsByType<SlimeController>(FindObjectsSortMode.None).Length >= maxAliveSlimes)  
        return;  
  
    Transform origin = spawnOrigin != null ? spawnOrigin : transform;  
    Vector3 spawnPos = origin.position + origin.forward * spawnDistance + Vector3.up * spawnHeight;  
  
    if (Physics.Raycast(spawnPos + Vector3.up * 30f, Vector3.down, out RaycastHit hit, 80f))  
        spawnPos = hit.point + Vector3.up * 0.55f;  
  
    Instantiate(slimePrefab, spawnPos, Quaternion.identity);  
}
```


SlimeSpawner

- Ajoutez SlimeSpawner sur l'objet PlayerManager.



SlimeSpawner

- Sur le prefab racine slime ajoutez SlimeController

